

Working with Actorium

This is a manual for the people doing the work — Product Owners, Tech Leads, and anyone reviewing or unblocking along the way. It also covers the two agents you'll interact with daily: the Coding Agent that does the implementation, and Hermes, the colleague agent you can just ask.

§1 roles at a glance

Who does what, and when

A feature moves through two human roles before any code gets written, and each role also approves something along the way — approving isn't a separate job, it's part of the role.

ROLE	PRODUCES	APPROVES
Product Owner	The product spec	The final feature handoff
Tech Lead	Technical design, then the task breakdown	The product spec (before design starts); task pull requests; unblocking stuck tasks

Two agents sit underneath both roles:

AGENT	DOES	SHOWS UP
Coding Agent	Claims a task, writes the code, opens a pull request	Automatically, once a task is ready
Hermes	Chats, answers questions, remembers context	Anytime you ask — not tied to a task



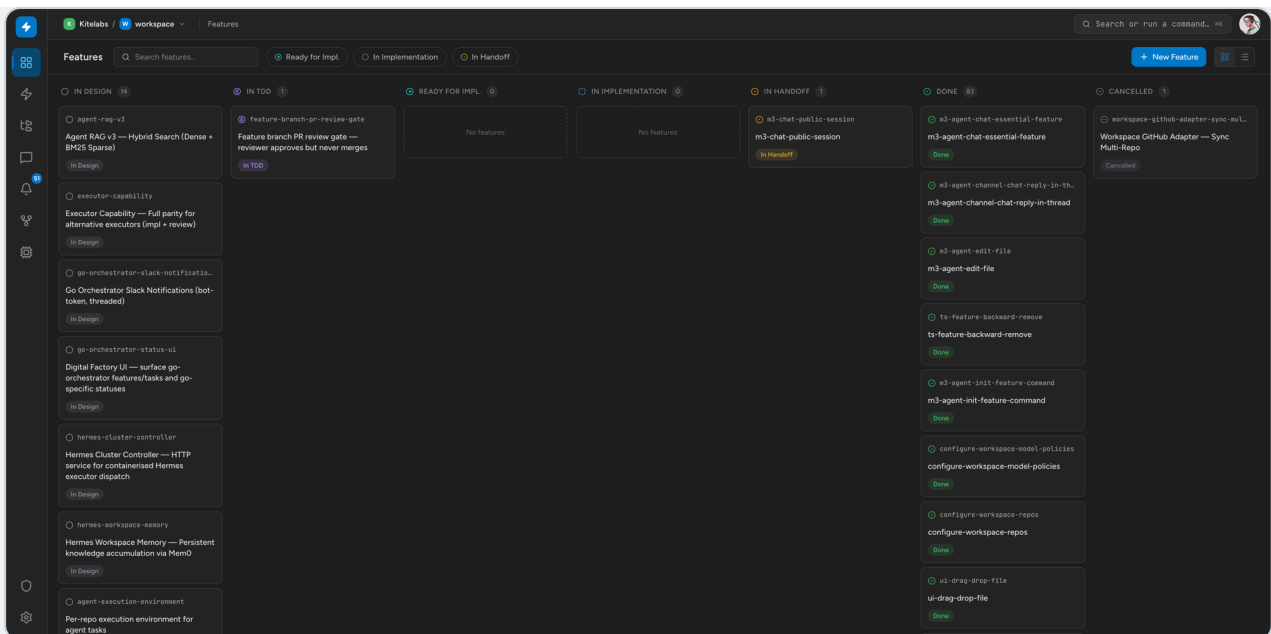


Fig. 1 The Features board — every feature as a card, grouped by its current stage: In Design, In TDD, Ready for Impl, In Implementation, In Handoff, Done, or Cancelled.

Who plays each role, today

Actorium is being introduced for two roles first — the rest of the team keeps working the way it already does until they're brought in.

ROLE IN THIS MANUAL	WHO THAT IS, TODAY	NOTE
Product Owner	Điện and Trung, together	Both are active while a feature is <code>in_design</code> — Điện brings the business ask, Trung turns it into the spec. Between them, that's the Product Owner role in this manual.
Tech Lead	Đạt	Unchanged from today — receiving the spec, estimating, breaking it into tasks, and approving along the way.

Design, progress reporting, and leadership sign-off aren't in Actorium yet — that work continues exactly as it does today, outside this tool.

How this maps to your process today

The stages you already run don't go away — Actorium tracks the two that are live right now and leaves the rest exactly where they are.

YOUR STAGE	WHO DOES IT	IN ACTORIUM TODAY
Tiếp nhận & thống nhất input/output	Điện + Trung	The feature is <code>in_design</code> — this is the Product Owner chapter.
Thiết kế UI/UX (nếu cần)	Tín	Not yet in Actorium — continues as today.
Soạn BRD/SRS, CTO xác nhận đủ input	Trung viết, Đạt xác nhận	The Tech Lead's "approve the product spec" step — the same gate as your Definition of Ready.
Phát triển & kiểm thử	Team Dev	Not yet in Actorium — continues as today.
Nghiệm thu nội bộ	Trung + Điện	This is the feature handoff — Product Owner approval, in our manual.
Trả bài Ban lãnh đạo	Ban lãnh đạo	Happens after Actorium marks the feature done — unchanged, outside the tool.

Two rules you already follow carry over exactly as they are:

- **"Không có input đủ – không bắt đầu"** is our `ready` state — a task can't start until its dependencies, and the Tech Lead's approval, are in place.
- **"Một yêu cầu – một ticket"** is our one-task, one-file rule — every unit of work has exactly one place it lives.

Product Owner

You start every feature and you're the one who confirms it actually shipped what you asked for.

What you produce

A product spec — a plain description of what's being built and why. It doesn't need implementation detail; that's the Tech Lead's job next. A good spec covers:

- The problem you're solving and who has it
- What "working" looks like, described concretely enough that someone could check it off
- What's explicitly out of scope, so nobody has to guess
- Any reference material — a design link, an example from elsewhere, a related past feature

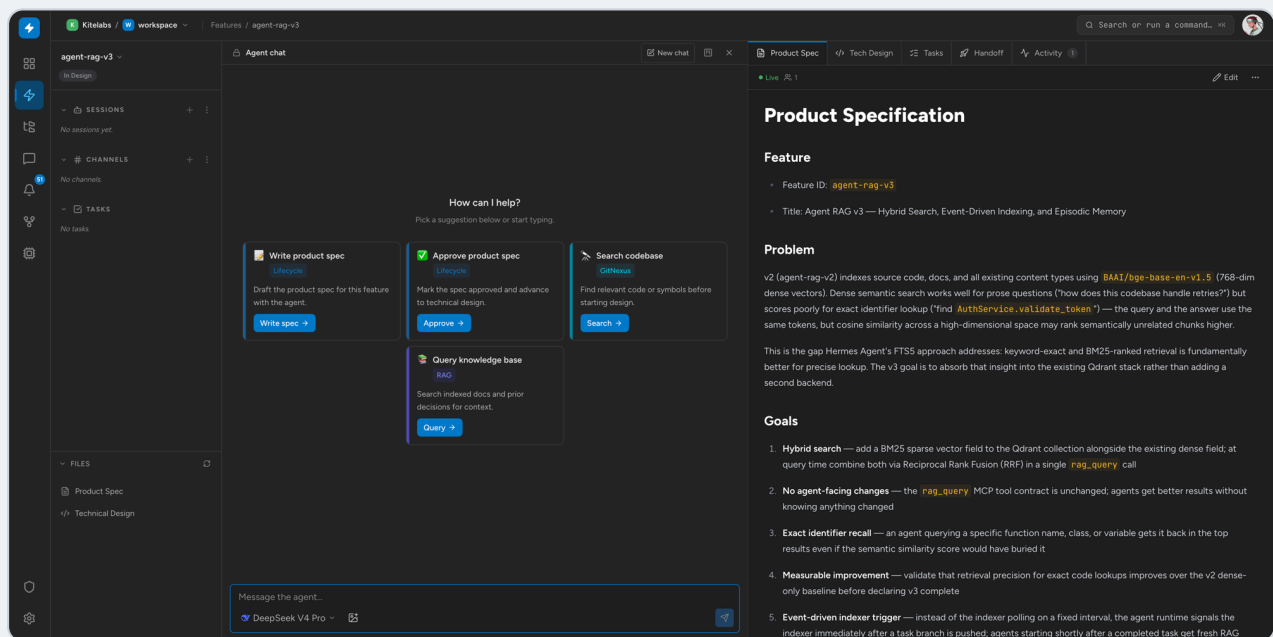


Fig. 2 A feature's Product Spec tab, with the agent chat open alongside it — "Approve product spec" here is what sends it on to the Tech Lead.

What happens after you submit

The Tech Lead reviews it for one thing: is this buildable as written. If they approve, it moves into technical design. If they reject it, you'll see why and can revise and resubmit — nothing is lost, it just goes back a step.

What you approve

You're not needed again until the very end. Once a feature reaches **handoff**, you check that what was actually built matches what you asked for in the spec. This is the last gate before a feature is marked done — if something's off, this is where you catch it.

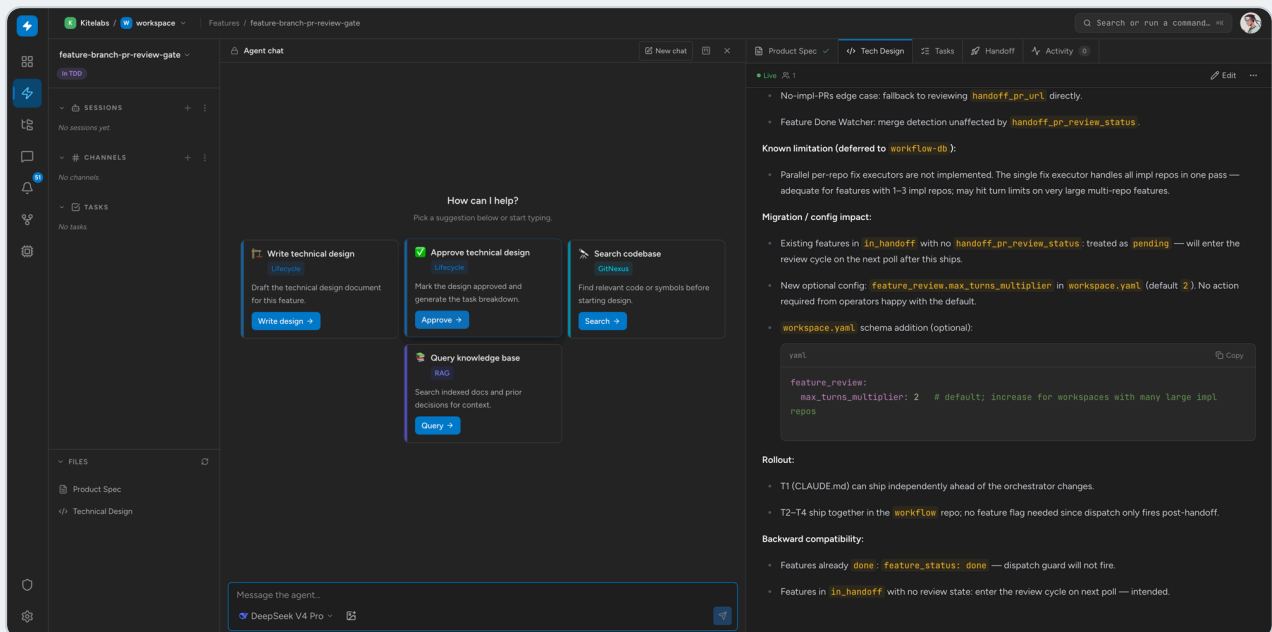


Fig. 3 The same feature workspace — the tab bar runs Product Spec, Tech Design, Tasks, Handoff, Activity. When a feature reaches the end, you'll use that same Handoff tab, with this same approve/reject pattern, to confirm what shipped.

Tech Lead

You turn an approved spec into a plan, and you're the approval gate at almost every step after that — the busiest role in the day-to-day.

What you produce, in order

1. **Technical design** — the approach: how the spec gets built, what changes where, and any tradeoffs worth flagging before committing engineering time.
2. **Task breakdown** — the design split into concrete units of work. Each task touches one part of the codebase and lists what it's waiting on, so it's clear at a glance what can start now versus what's blocked on something else finishing first.

```
text Copy

T1: management-repo - CLAUDE.md: add `reviewing` status + update transitions
  └─ Can begin now - no blockers

T2: workflow - dispatchFeaturePrReviewer: poll step + status.yaml schema + reviewer briefing
  └─ Can begin now - no blockers
  └─ T1 and T2 run in parallel

T3: workflow - dispatchFeaturePrFixExecutor: single fix executor dispatch + post-fix reset
  └─ BLOCKED on T2 (fix dispatch triggered by review outcomes written in T2's reap path)

T4: workflow - unit + integration tests for T2 and T3
  └─ BLOCKED on T2 (reviewer dispatch must be implemented)
  └─ BLOCKED on T3 (fix executor dispatch must be implemented)
```

Fig. 4 A real task breakdown — each task lists what it's blocked on, so it's clear at a glance what can start now (T1, T2) versus what's waiting on something else first (T3, T4).

What you approve

- **The product spec**, before you start designing against it — you're checking it's buildable, not re-opening the product decision itself.
- **Task pull requests**, once a Coding Agent's work has passed automated review and is waiting on a human to merge it.
- **Unblocking stuck tasks** — deciding what happens next when an agent hits something it can't resolve on its own (a missing credential, an ambiguous requirement, a test it couldn't fix after several tries).

NOTE

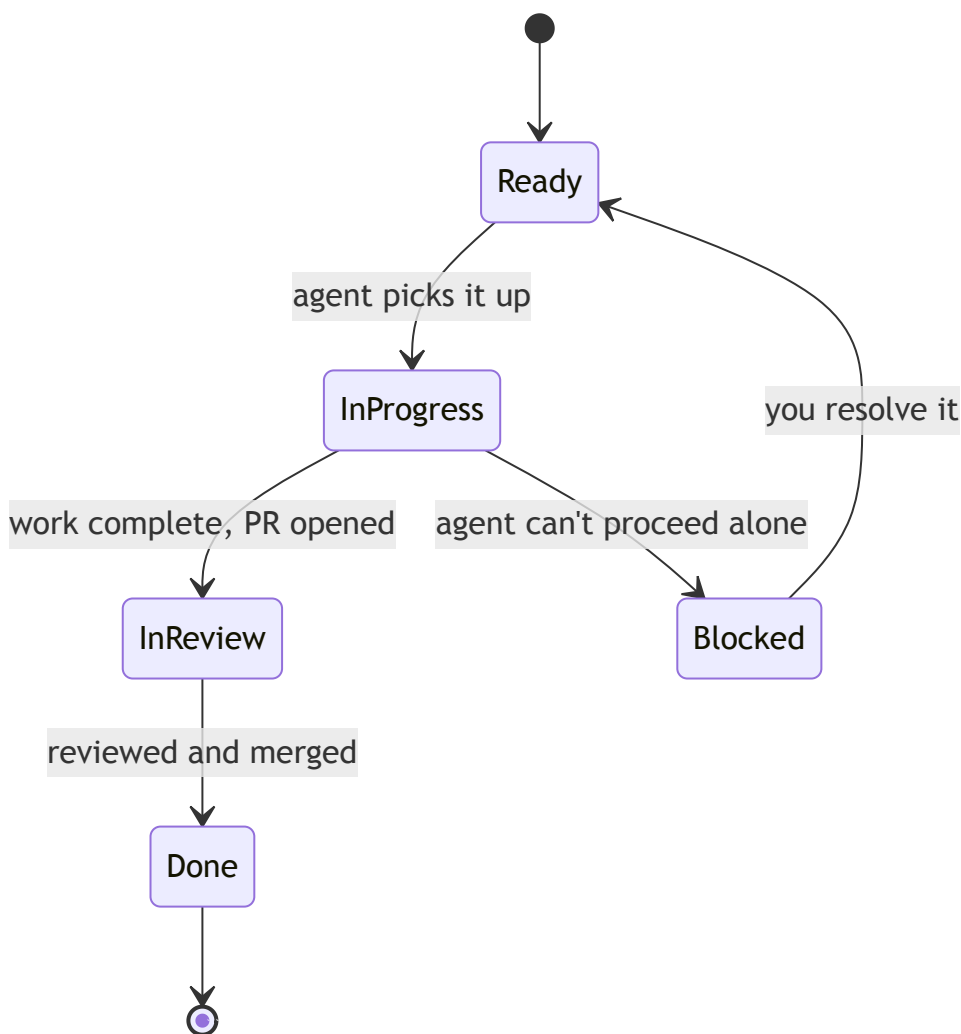
In a smaller team you may hold both the Product Owner and Tech Lead role yourself — the approvals still happen, you're just the one doing both sides of the handoff.

The agents you work alongside

Neither of these reads a manual — this chapter is for you, covering what to expect from each one.

The Coding Agent

Once a task is ready — its dependencies are done and a human has approved the plan behind it — a Coding Agent picks it up automatically. You don't assign it by hand.



What you'll see a task's status do, from the moment it's ready.

What lands in the pull request: a description of what changed and why, and normally a passing review. If something's wrong, you won't see a silent failure — the task moves to **blocked** with a plain explanation and a suggestion for what to do about it.

Hermes, the colleague agent

Hermes isn't tied to a task — talk to it the way you'd ask a teammate a question, in chat or a thread. It remembers the context of your conversation and the project's own specs and decisions, so you don't have to re-explain background every time.

- **Good for:** status questions, "why did we decide this," drafting a quick answer, catching you up after time away.
- **Not for:** actually shipping a code change — that still goes through a task and a Coding Agent.

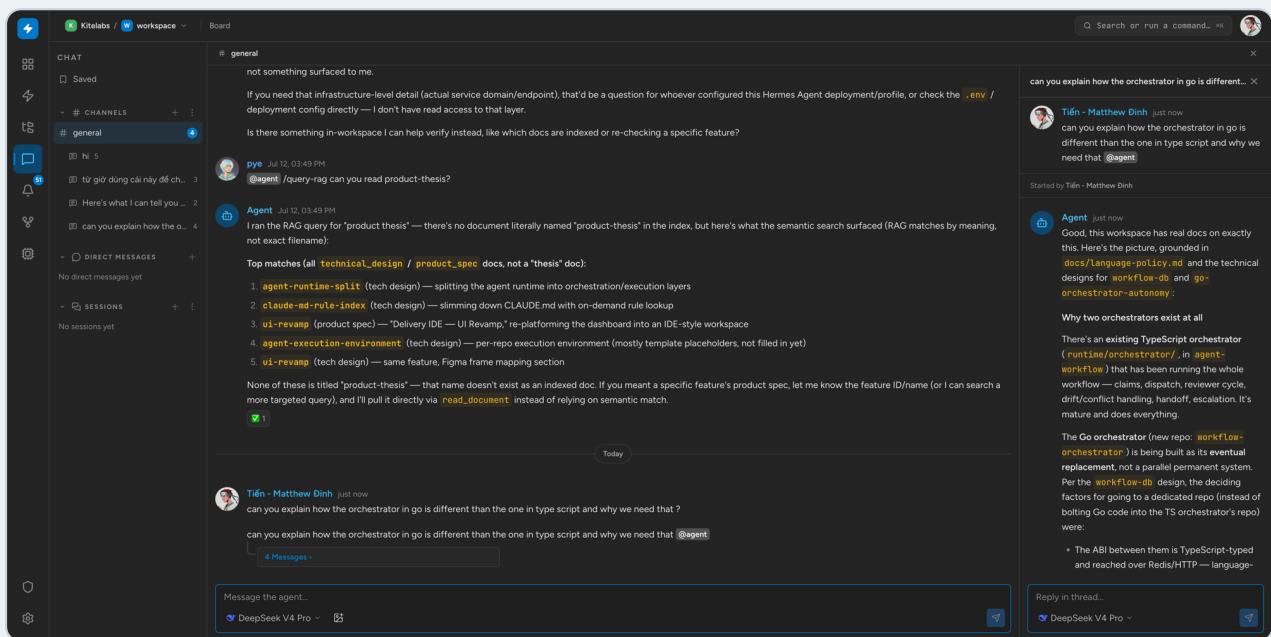


Fig. 5 A real conversation with Hermes in team chat (labeled "Agent" in this UI) — including a threaded follow-up where it answers by citing the workspace's own documents instead of guessing.

Common walkthroughs

Starting a new feature end to end

- 1 Product Owner writes and submits the product spec.
- 2 Tech Lead approves it, then writes the technical design.
- 3 Tech Lead breaks the design into tasks and gets it approved.
- 4 Tasks whose dependencies are already met turn `ready` automatically — Coding Agents start picking them up without anyone dispatching them by hand.

Unblocking a stuck task

- 1 You see a task marked `blocked`, with a reason and a suggested next step attached.
- 2 You take the action needed — for example, providing a missing piece of information or resolving an external dependency.
- 3 You mark it resolved; the task resumes from where it left off.

Reviewing and merging a passed task

- 1 A task's pull request passes automated review and sits waiting for you.
- 2 You look over the change.
- 3 You merge it — the task moves to `done`.

Asking Hermes a question mid-project

- 1 Open the chat and ask in plain language — no special syntax needed.
- 2 Hermes answers using the project's own specs, designs, and history as context.

- -
 - 3 If the answer means something needs to change in the product, turn that into a task rather than expecting Hermes to make the change itself.
-

Glossary

Feature

One piece of product work, from spec through to shipped and approved.

Task

One concrete unit of work inside a feature, scoped to a single part of the codebase.

Dependency

Another task that has to finish first — a task waits until everything it depends on is done.

Ready

A task whose dependencies are all done and can now be picked up.

Blocked

A task an agent couldn't finish alone — it's waiting on a human decision or action.

Review passed

A task's work has been checked and approved, and is waiting on a human to merge it.

Handoff

The final check at the end of a feature, where the Product Owner confirms what shipped matches what was asked for.

Done

A feature or task that's fully finished — nothing further is expected.